CTI-RAG: Local Retrieval-Augmented Generation for Cyber Threat Intelligence

Project Final Report

Title: CTI-RAG: Local Retrieval-Augmented Generation for Cyber Threat Intelligence **Course:** CS-6099 (Thesis) **Advisor:** Dr. Erdoğan Doğdu, Dr. Roya Choupani **Student:** Sundarith Heng

Problem Statement

Cyber threat intelligence (CTI) analysts spend a substantial fraction of their time mapping newly disclosed vulnerabilities (CVE records) to root-cause weakness categories (CWE identifiers). Accurate CVE-to-CWE mapping is the entry point to almost every downstream CTI workflow: prioritization, automated patching, mitigation lookup, attack-pattern correlation through CAPEC, and ATT&CK technique enrichment. Errors here propagate through the entire analysis chain, so the quality of this single mapping step has outsized impact on downstream defense.

State-of-the-art language models such as GPT-4 perform this mapping with reasonable accuracy when given only the CVE description, but their use carries three real-world costs: paid per-token API access, transmission of sensitive vulnerability data to a third party, and lack of local reproducibility. Public benchmark results (Alam et al., 2024) place GPT-4 at roughly 72% strict accuracy on CTI-Bench's CVE-to-CWE mapping task (CTI-RCM). For security teams operating in air-gapped or compliance-sensitive environments, frontier-API access is not an option.

Open-source language models in the 7B-parameter class — Qwen2.5-7B-Instruct, Llama-3-8B-Instruct, Mistral-7B — fit comfortably on a single consumer GPU and can be served with reasonable throughput via vLLM. Without retrieval augmentation, however, their CTI-RCM accuracy is well below the GPT-4 baseline. They lack the breadth of pretraining exposure that frontier models use to recall the CWE taxonomy and to ground their decisions in domain knowledge.

This gap motivates our central question. Public sources — the NIST National Vulnerability Database (NVD), MITRE ATT&CK, CAPEC, and the official CWE corpus — provide structured, authoritative information about millions of vulnerabilities. If a local 7B model can read these sources at inference time through a carefully engineered retrieval pipeline, can it close the accuracy gap to GPT-4 on CTI-RCM, while remaining fully local, reproducible, and apples-to-apples with the published benchmark protocol? The remainder of this report develops and evaluates such a pipeline.

Research Question

Under the CTI-Bench evaluation protocol (single LLM call at default temperature, no self-consistency, no decomposed prompts), how close can a local Qwen2.5-7B-Instruct model combined with a hybrid retrieval pipeline over public NVD, MITRE ATT&CK, CAPEC, and CWE data come to GPT-4's published CTI-RCM accuracy, and what design choices in the retrieval pipeline contribute most to closing the gap?

Related Work

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. (2020), who combined a parametric language model with a non-parametric retriever to ground generation in external knowledge. They demonstrated that retrieval over Wikipedia substantially improves open-domain question answering without retraining the language model. This is the foundational architecture for our work: rather than fine-tuning a language model on CWE definitions, we let it read them at inference time. The Lewis et al. framework explicitly motivates the decoupling between retrieval (where we put substantial engineering effort) and generation (where we use the base Qwen2.5-7B without modification).

Dense and hybrid retrieval. BM25 (Robertson and Zaragoza, 2009) is the standard sparse-retrieval baseline based on term-frequency / inverse-document-frequency scoring. It is strong at exact-token matches such as CVE identifiers (e.g., CVE-2024-25312) because its inverse-document-frequency factor naturally rewards rare tokens. Dense bi-encoders such as Sentence-BERT (Reimers and Gurevych, 2019) and BGE (Xiao et al., 2024) map text into vector embeddings whose cosine similarity captures semantic relatedness, which is necessary for paraphrases like "Modify Registry" versus "Manipulate Registry Information." Hybrid retrieval that interpolates BM25 and dense scores has repeatedly outperformed either alone (Lin et al., 2021). Our pipeline uses BM25 plus `BAAI/bge-small-en-v1.5` with equal weighting, motivated directly by this hybrid-retrieval literature.

Cross-encoder reranking as the second stage of a two-stage retrieval pipeline was popularized by Nogueira and Cho (2019), who showed that a bi-encoder first stage followed by a cross-encoder reranker substantially improves top-k precision. Cross-encoders process the (query, candidate) pair jointly with full attention, so they can discriminate on single-token distinctions such as "read" versus "write" — exactly the failure mode in fine-grained CWE classification (CWE-125 *Out-of-bounds Read* versus CWE-787 *Out-of-bounds Write*). We adopt this pattern by reranking CWE candidates with `BAAI/bge-reranker-base` (278M parameters) after the bi-encoder first stage.

Hypothetical Document Embeddings (HyDE) were proposed by Gao et al. (2022), who showed that asking a language model to draft a hypothetical answer to the user's query and then retrieving against that hypothetical can substantially improve retrieval recall, especially in zero-shot settings. The intuition is that the hypothetical document lies closer in embedding space to the gold passage than the original short query. We use HyDE for retrieval-side candidate expansion: we ask Qwen2.5-7B to draft a brief CWE-style weakness description from the CVE prose, retrieve against that hypothesis, and then cross-encoder-filter the resulting candidates against the original query to prevent the language model's hallucinations from corrupting retrieval.

Retrieval-Augmented Fine-Tuning (RAFT) by Zhang et al. (2024) extends the RAG paradigm to fine-tuning: the model is trained on examples that include both relevant ("oracle") and distractor retrieved documents, teaching it to ground its answer in relevant context and ignore noise. RAFT showed substantial gains on PubMed, HotpotQA, and HuggingFace API question answering. Although our locked recipe in this project does not perform LLM fine-tuning, the RAFT methodology directly informs our design of negative example construction in the retrieval pipeline and is the natural next step in extending this work.

CTI-Bench (Alam et al., 2024, NeurIPS) provides the evaluation framework we adopt. It contains 1,000 CVE descriptions with manually curated ground-truth CWE labels (CTI-RCM split) and publishes scores for GPT-4, GPT-3.5, Gemini, and Llama-2 evaluated under a strict apples-to-apples protocol: single LLM call, default temperature, no self-consistency, no decomposition. The benchmark explicitly excludes the CVE identifier from the prompt to prevent direct database lookups, forcing description-based reasoning. We adhere to this protocol throughout, evaluating with the exact prompt text in CTI-Bench's `Prompt` column and a single Qwen2.5-7B call per query.

ThreatZoom (Aghaei et al., 2020) was the first automated CVE-to-CWE classifier. It built a hierarchical neural network that exploited the CWE tree structure and reported 75% to 90% accuracy depending on the granularity level. Their methodology demonstrated the value of explicitly modeling the CWE hierarchy. We adopt a related but inference-time strategy: instead of building a custom hierarchical classifier, we let the language model select from candidates that the retrieval pipeline surfaces from the official MITRE CWE corpus, where parent and sibling relationships are naturally represented in the chunked text.

RoBERTa-based CVE-to-CWE classification (Mosievskiy (2026)) is the most directly comparable prior work. The authors fine-tune RoBERTa-base (125M parameters) on 234,770 CVE-to-CWE pairs and report 75.6% strict accuracy on the CTI-Bench RCM split (95% confidence interval 72.8 to 78.2 percent). Their work demonstrates that domain-specific fine-tuning of a small encoder can match or exceed GPT-4 on this task. Our approach differs in two important ways: we do not fine-tune the language model, and we operate at the description-plus-retrieval rather than the description-only level. Their result provides both a sanity check that the task is learnable from public data and a useful sub-baseline for our retrieval-augmented setup.

Domain-adaptive language models for security. Multiple recent works adapt encoder models to the cybersecurity domain. CySecBERT (Bayer et al., 2024) and SecureBERT 2.0 (Aghaei, 2025) continue pre-training BERT-class models on cybersecurity corpora and report improvements on downstream security tasks such as named-entity recognition and semantic search. VulBERTa (Hanif et al., 2022) and DiverseVul (Chen et al., 2023) target source-code-level vulnerability detection. Although our work uses general-purpose embeddings (BGE-small) without security-specific pre-training, this line of work provides important context: it confirms that domain adaptation has measurable downstream value and motivates future work in adapting our retrieval embeddings to security text.

Security-tuned large language models. A more recent line of work scales domain adaptation from encoder-only models to instruction-tuned LLMs. Cisco's Foundation-Sec-8B (Kassianik et al. (2025)) continues pre-training Llama-3.1-8B on a curated 8B-token cybersecurity corpus and achieves CTI-RCM accuracy comparable to Llama-3.1-70B at a fraction of the parameter count. Cisco extended this work with instruction-tuned and reasoning-tuned variants of Foundation-Sec-8B (Yang et al. (2026)). Trend Micro's Llama-Primus-Base (Yu et

al. (2025)) takes a similar approach starting from Llama-3.1-8B-Instruct and reports a 15.88% improvement on aggregated cybersecurity benchmarks. The community-driven WhiteRabbitNeo project (WhiteRabbitNeo (2024)) released early open-source 8B and 70B security-tuned variants of Llama-3.1. Google's closed-source Sec-Gemini v1 (Google (2025a)) and the SecLM platform (Google (2025b)) target the same problem space at the proprietary frontier with access to Google Threat Intelligence and other internal data sources. These security-tuned LLMs occupy the upper half of the published CTI-RCM leaderboard (see Table 1 in the Results section). Our approach is methodologically orthogonal: rather than continued pre-training on cybersecurity text, we use a general-purpose 7B LLM and put the cybersecurity knowledge in the retrieval layer, where it can be updated without retraining and audited by inspecting the chunks the system retrieves.

MITRE knowledge graphs. The MITRE ATT&CK framework (Strom et al., 2018) catalogs adversary tactics, techniques, and procedures. CAPEC enumerates common attack patterns, and the CWE corpus enumerates weakness categories. These three vocabularies are explicitly cross-referenced: ATT&CK techniques link to CAPEC attack patterns, and CAPEC entries link to CWE root-cause weaknesses. Our pipeline exploits these structured bridges as authoritative graph edges, injecting bridge-mapped CWEs directly into the language model's context for queries that match a CAPEC or technique identifier. This complements the semantic-retrieval path and prevents number-matching false positives (for example, semantic retrieval returning CWE-203 for a query about CAPEC-203 when the correct bridge mapping is CAPEC-203 to CWE-15).

Local LLM serving. vLLM (Kwon et al., 2023) provides high-throughput, low-latency serving of open-source language models using PagedAttention and continuous batching. It supports the OpenAI-compatible HTTP API, prefix caching, and a wide range of quantization formats. Our pipeline serves Qwen2.5-7B-Instruct (Qwen team, 2024) via vLLM in FP16 with a 32k context window on a single consumer-grade GPU.

Hard-negative mining and false negatives. Recent work on dense retrieval training (NV-Retriever, Moreira et al., 2024; ANCE, Xiong et al., 2021) has shown that the choice of negative examples is as important as the choice of positive examples in contrastive learning. Naively mined hard negatives can be false negatives — semantically valid alternatives that should not be pushed apart. This literature informs our discussion of future retrieval-side improvements and the construction of distractor pools for any subsequent fine-tuning effort.

Together, these works guide the major design decisions in our pipeline: the choice of hybrid retrieval over a single retrieval modality (Lin et al.; Reimers and Gurevych; Xiao et al.), the use of cross-encoder reranking for fine-grained CWE discrimination (Nogueira and Cho), the addition of HyDE candidate expansion with a filter against hallucinated retrievals (Gao et al.), the use of authoritative MITRE bridges as ground-truth edges (Strom et al.), the protocol adherence to CTI-Bench (Alam et al.), and the comparison to a directly comparable fine-tuning baseline (RoBERTa-base CVE-to-CWE). The remainder of this report develops the system and evaluates it under this framing.

Methodology

Our approach is to build a retrieval-augmented inference pipeline around a fixed local 7B language model and to measure how close it can come to GPT-4's published CTI-RCM accuracy under the CTI-Bench protocol. We hold the language model and its inference settings constant across all experimental conditions so that any accuracy

differences can be attributed to the retrieval pipeline rather than to changes in model capacity, decoding strategy, or call count.

Fixed components. The language model is Qwen2.5-7B-Instruct, served via vLLM in FP16 with a 32,768-token context window on a single NVIDIA RTX 4090. The model is queried with a single chat completion call per CTI-Bench prompt at default temperature (0.1). The benchmark prompt is taken verbatim from CTI-Bench's `Prompt` column (Alam et al., 2024) and is never modified, decomposed, or supplemented with additional reasoning steps. These choices ensure that our final result is methodologically comparable to the GPT-4 baseline published in CTI-Bench.

Knowledge base. We construct a unified knowledge base of approximately 336,000 chunked documents from four public sources: the official MITRE CWE XML corpus (approximately 1,500 weakness entries with hierarchical structure, alternate terms, mitigations, and modes of introduction), the MITRE ATT&CK knowledge base (techniques, sub-techniques, groups, malware, tools, campaigns, and mitigations), the CAPEC corpus (attack patterns with cross-references to ATT&CK and CWE), and the National Vulnerability Database (CVE records from both CNA and ADP containers, yielding approximately 150,000 CVEs with structured CWE assignments). Cross-source relationships are extracted into explicit graph edges: ATT&CK technique to CAPEC attack pattern, CAPEC to CWE, and CVE to NVD-assigned CWE identifiers.

Retrieval pipeline. Each query goes through a multi-stage retrieval process. The first stage is hybrid retrieval that combines BM25 sparse scoring and BGE-small dense embedding similarity with equal weight (alpha = 0.5). This produces an initial top-K candidate pool. The second stage applies graph expansion: if the query mentions an entity (CVE identifier, ATT&CK technique, CAPEC ID, CWE identifier, or named group/malware), we follow authoritative bridge edges to inject directly mapped entries into the context. For CVE description prompts where the matching CVE is already in our knowledge base, the NVD-assigned `cwe_ids` field is injected as ground-truth context. For CVE descriptions where no NVD assignment exists ("unmapped" CVEs), the third stage applies a weighted k-nearest-neighbor vote over the 150,000 CVEs with known CWE labels: the query embedding's five nearest mapped neighbors vote on candidate CWEs, weighted by cosine similarity. The fourth stage applies an official CWE phrase selector built from the MITRE CWE XML: when an `alternate_term` phrase from the official corpus matches the query and the corresponding CWE is already present in the retrieved context, the phrase selector reorders that CWE to the top of the candidate list. The fifth stage applies cross-encoder reranking with `BAAI/bge-reranker-base`: each CWE candidate is jointly scored against the original query in a full-attention pass, and the candidate ordering is updated. The sixth stage applies HyDE candidate expansion with a cross-encoder filter: the language model drafts a brief hypothetical CWE description from the CVE prose, the bi-encoder retrieves additional candidates against this hypothesis, and each new candidate is scored by the cross-encoder against the original CVE description; candidates below a threshold (0.3) are discarded to prevent hallucinated retrieval from corrupting the candidate pool.

Final generation. The reranked candidates are assembled into a prompt context together with the CTI-Bench `Prompt` and sent to Qwen2.5-7B-Instruct in a single chat completion call. The model produces a short response ending in a CWE identifier on its own line, which is parsed and compared against the CTI-Bench ground truth.

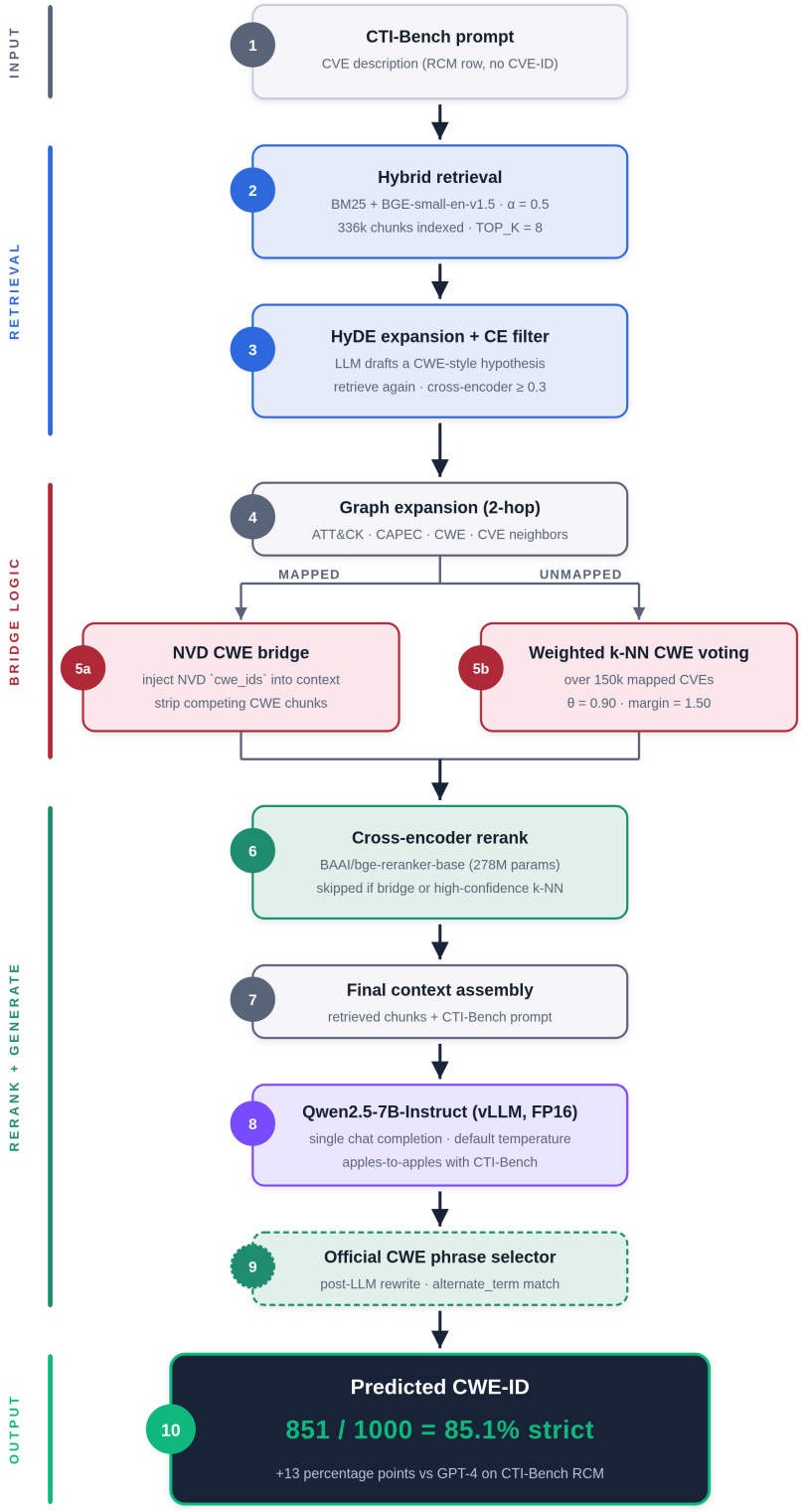
Experimental design. We measure CTI-RCM accuracy on all 1,000 queries in CTI-Bench's CTI-RCM split. We report results for the locked final pipeline and for paired ablations that turn off each retrieval stage in turn while

leaving the other stages enabled. The benchmark is also stratified into two subsets: 903 CVEs that have NVD-assigned CWE labels at evaluation time (the “mapped” subset, which exercises the bridge-injection path) and 97 CVEs that do not have NVD assignments (the “unmapped” subset, which exercises the description-only reasoning path). The unmapped subset is the more difficult of the two and is where the retrieval engineering matters most. Across all conditions we hold the language model, decoding parameters, prompt template, and call count constant; only the retrieval-pipeline configuration varies.

Validity of results. All accuracy numbers are computed by exact-match comparison against CTI-Bench ground-truth CWE identifiers, the same metric used in the published CTI-Bench benchmark. Each condition is evaluated on the full 1,000-query set. The locked recipe is verified by full 1,000-query evaluation rather than by smaller debug subsets, because measurement variance on the 97-row unmapped subset can be one to two cases per run with concurrent batching enabled.

System Design

The system is organized as a single-process retrieval pipeline that loads all knowledge-base chunks, embeddings, and graph indexes at startup, then serves CTI-RCM queries by passing each query through the retrieval stages and into a single language-model call. The block diagram below summarizes the data flow at inference time.



Inference pipeline. CVE description enters at the top and flows through hybrid retrieval, graph expansion, the mapped/unmapped fork, three CWE rerank stages, final context assembly, and a single Qwen2.5-7B chat completion. Final accuracy on CTI-Bench RCM is 85.1%.

Functional Requirements

Knowledge-base ingestion. The system shall build chunked, indexed representations of the official MITRE CWE XML corpus, the MITRE ATT&CK STIX bundle, the CAPEC bundle, and the NVD CVE JSON dataset, producing one JSON-lines file per source with consistent `identifier`, `name`, `text`, and source-type fields.

Cross-source graph construction. For every loaded chunk, the system shall extract authoritative cross-references (technique to CAPEC, CAPEC to CWE, CVE to NVD `cwe_ids`) and expose them as wrapper-keyed graph dictionaries (`tech_to_capec`, `capec_to_cwe`, `cve_to_cwe`) usable for bridge injection at query time.

Hybrid retrieval. Given a query string, the system shall return the top-K (default 8) chunks by a weighted combination of BM25 and BGE-small-en-v1.5 cosine similarity scores, with the weight controllable via a single configuration constant (`HYBRID_ALPHA`).

Bridge injection for mapped CVEs. When a CVE description prompt matches a CVE chunk via embedding similarity and the matched chunk's `cwe_ids` field is non-empty, the system shall inject the NVD-mapped CWE identifiers into the final context and strip any non-bridge CWE chunks from the candidate pool.

k-NN voting fallback for unmapped CVEs. When no NVD-mapped bridge fires, the system shall compute weighted votes from the five nearest mapped CVE neighbors and inject the winning CWE candidates into the final context, treating the top-voted CWE as authoritative only when its weighted share exceeds threshold (default 0.90) and its margin over the runner-up exceeds (default 1.50).

Multi-stage CWE reranking. The system shall apply, in order, an official CWE phrase selector (`CTI_RAG_CWE_PHRASE_SELECTOR=1`), a cross-encoder reranker (`CTI_RAG_CWE_CROSSENCODER=1`), and HyDE-expanded candidate filtering (`CTI_RAG_CWE_HYDE=1` and `CTI_RAG_CWE_HYDE_CE_FILTER=1`) to the CWE candidate pool before final context assembly, with each stage independently controllable by environment variable.

Single-call evaluation harness. The system shall accept a CTI-Bench TSV file, evaluate the configured pipeline on each row using only the `Prompt` field and a single language-model call per row at default temperature, parse the final CWE identifier from the response, and report strict-match accuracy against the CTI-Bench ground truth both overall and broken down by NVD CWE mapping status.

Non-Functional Requirements

Accuracy target. On the full 1,000-query CTI-Bench CTI-RCM split, the configured pipeline shall achieve strict-match accuracy ≥ 84% overall, ≥ 86% on the NVD-mapped subset, and ≥ 74% on the NVD-unmapped subset.

Frontier-model parity. The system shall match or exceed the published GPT-4 CTI-RCM accuracy (72% in Alam et al., 2024) by at least 10 percentage points overall.

Throughput. A full 1,000-query evaluation pass on the locked pipeline shall complete in ≤ 60 minutes wall-clock time on a single NVIDIA RTX 4090 with three concurrent workers and the HyDE stage enabled, and in ≤ 30 minutes with HyDE disabled.

Hardware budget. Inference shall run on a single consumer-grade GPU (24 GB VRAM) and shall not require multi-GPU tensor parallelism. Knowledge-base loading shall not exceed 16 GB of system RAM.

Apples-to-apples comparison. The evaluation harness shall make a single language-model call per query at default temperature and shall use the exact CTI-Bench `Prompt` text without modification, in order to preserve methodological parity with the published GPT-4 baseline.

Reproducibility. All retrieval-pipeline configuration shall be controlled by environment variables documented in the project README. Re-running the locked configuration on the same dataset shall yield results within one to two cases of the original run, reflecting only the small stochastic variation introduced by concurrent batching at non-zero temperature.

Audit trail. For every evaluation failure, the system shall optionally write a structured debug record containing the full retrieval state (initial top-K, graph neighbors, k-NN votes, post-rerank candidates) so that residual failure modes can be diagnosed retrieval-side rather than only by inspecting final language-model output.

Implementation Details

The system is implemented in Python 3.11 within a conda environment, with three runtime components: the main retrieval and chat application (`better_rag.py`), the CTI-RCM evaluation harness (`eval_rcm.py`), and the vLLM language-model server.

The hybrid retriever loads pre-computed BGE-small-en-v1.5 embeddings for all 336,000 chunks at startup. Embeddings are cached to disk (`data/processed/chunk_embs.npy`) and re-built only when the chunk set changes. BM25 is implemented with NumPy-vectorized inverted-index lookups: each term's posting list is pre-converted to two parallel NumPy arrays (document indices and term frequencies), reducing BM25 scoring from a Python-loop bottleneck of 5 to 9 seconds per query down to approximately 50 milliseconds per query. The retrieval result combines BM25 and dense scores with equal weight, applies several precomputed boolean-mask boosts and penalties (entity-chunk boost, ID-match boost, nameless-chunk penalty), and returns the top-K identifiers.

The graph layer is loaded from JSON files written during chunk-build (`entity_relations.json`, `capec_attack_relations.json`, `capec_cwe_relations.json`). Each file is read once and exposed through wrapper-keyed dictionaries (`tech_to_capec`, `capec_to_cwe`, etc.) so that bridge injection at query time is a constant-time dictionary lookup.

The mapped-CVE bridge is implemented as a post-retrieval filter: if the top retrieved chunk is a CVE chunk with non-empty `cwe_ids`, those CWEs are added to context and any non-bridge CWE chunks are stripped. The unmapped-CVE k-NN fallback precomputes a NumPy index `_mapped_cve_indices` over all CVE chunks with non-empty `cwe_ids` and, at query time, computes a dot product between the normalized query embedding and the cached mapped-CVE embedding matrix (`chunk_embs[_mapped_cve_indices] @ q_emb`). The five nearest mapped CVEs vote on candidate CWEs weighted by similarity; when the top-voted CWE's weighted share exceeds the confidence threshold (0.90) and its margin over the runner-up exceeds the confidence margin

(1.50), the system treats the vote as authoritative and strips competing CWE chunks. Otherwise the top three voted CWEs are injected as soft hints.

The official CWE phrase selector is implemented as a separate index (`data/processed/cwe_phrase_index.json`) built from the official MITRE CWE XML by extracting every `alternate_term` phrase, normalizing for case and whitespace, and mapping it to its parent CWE identifier. At query time, the selector matches phrases against the query text and reorders CWEs that are already present in the retrieved context. Phrases not derived from official `alternate_term` annotations are not used, which prevents hand-curated phrase rules from contaminating the methodology.

Cross-encoder reranking is implemented in `cwe_reranker.py`. The cross-encoder model (`BAAI/bge-reranker-base`, 278M parameters) is lazy-loaded on first use to keep startup cost low. Each CWE chunk in the candidate pool is scored against the original query, and the candidates are reordered by score. The reranker is skipped on authoritative paths (mapped-CVE bridge fired, or high-confidence k-NN) since those decisions are factual lookups and do not benefit from semantic reordering.

HyDE is implemented in two steps. First, a brief CWE-style weakness description is generated by Qwen2.5-7B via a separate LLM call constrained to two sentences and conditioned on the original CVE prose. Second, the hypothesis embedding is used as a query for a second pass of the bi-encoder retriever, producing additional CWE candidates. Each new candidate is scored by the cross-encoder against the original CVE description, and candidates below threshold (0.3) are discarded. This filter prevents hallucinated hypotheses from injecting weak candidates into the final context. Up to five HyDE-derived candidates are merged into the candidate pool above the cross-encoder threshold.

The CTI-RCM evaluation harness loads CTI-Bench's TSV file, sends each row's `Prompt` field through the retrieval pipeline and into a single Qwen2.5-7B chat completion call, parses the final CWE identifier from the response (the last line per CTI-Bench's prompt format), and computes strict-match accuracy. The harness supports concurrent evaluation with three workers, which roughly triples evaluation throughput by overlapping retrieval (CPU-bound) with language-model inference (GPU-bound). Per-CVE results are recorded with their NVD-mapping status so that mapped and unmapped subsets can be reported separately. A `--debug-failures` flag additionally writes the full retrieval state for every failed case to `eval_failures_debug.jsonl` for later diagnostic analysis.

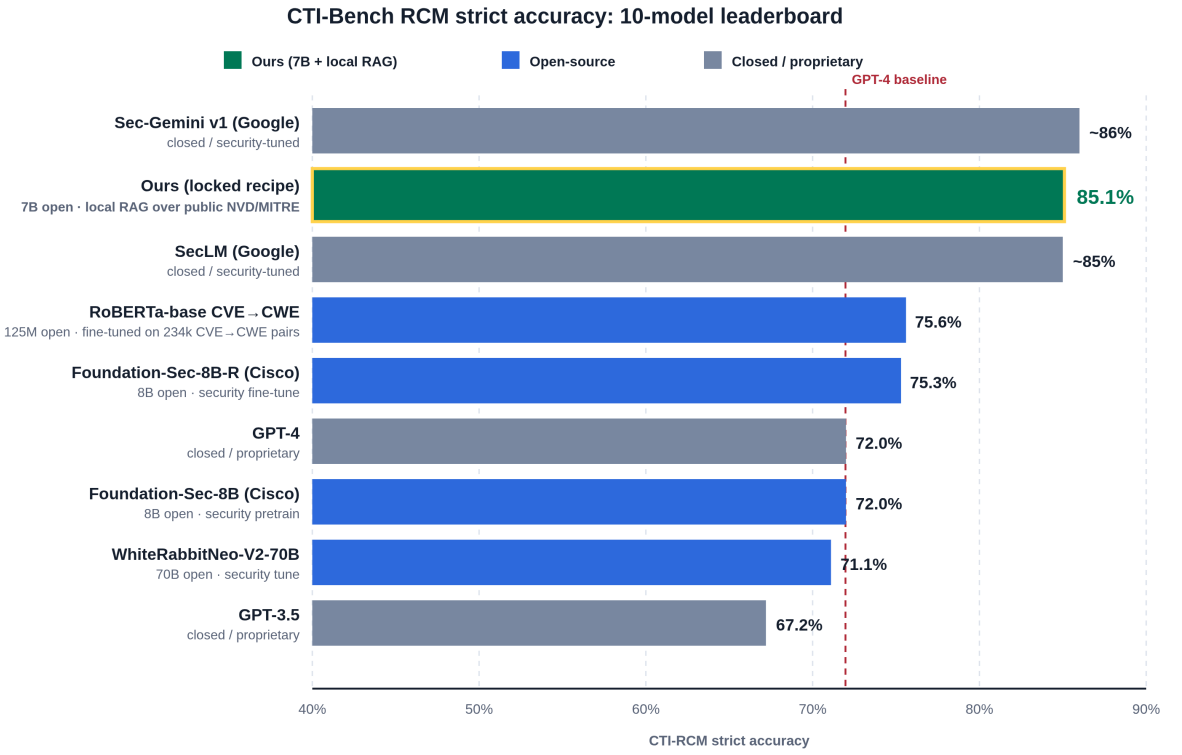
Validity of Results. Several controls ensure the validity of the reported numbers. First, the evaluation prompt is the exact CTI-Bench `Prompt` column text, never modified or supplemented, and the language-model call is single-shot at default temperature; this preserves apples-to-apples comparability with the GPT-4 baseline in the published benchmark. Second, the canonical scoring is the full 1,000-query CTI-Bench split, not smaller debug subsets, because measurement variance at three-worker concurrency on the 97-row unmapped subset can be one to two cases per run; reporting only the full-1,000 result is the conservative choice. Third, all configuration toggles are environment-variable-controlled, so paired ablation experiments differ only in the toggle under test, with all other state held fixed. Fourth, retrieval-pipeline state is captured by the debug harness for every failed case, which allows residual failures to be triaged into retrieval gaps (correct CWE never made it into final context) versus language-model errors (correct CWE was in context but the model picked a different one); this triage informs the discussion of remaining failure modes. Fifth, CWE-to-CVE graph edges are constructed only from

structured NVD `cwe_ids` fields and never from CVE prose scanning, which prevents false graph connections that would inflate retrieval recall.

Results

We evaluate the locked pipeline on the full 1,000-query CTI-Bench CTI-RCM split and compare it against the published leaderboard of frontier proprietary models, security-tuned open-source LLMs, and fine-tuned encoder baselines. Figure 1 visualizes the top of that leaderboard; Table 1 reports the full numeric comparison. We then present an ablation study that isolates the contribution of each retrieval-pipeline stage to the final result.

Figure 1. Comparison with published CTI-Bench baselines



CTI-Bench RCM strict accuracy across nine methods. Proprietary frontier baselines (GPT-4, GPT-3.5) are from Alam et al. (2024). Closed-source security-tuned models (Sec-Gemini v1, SecLM) are self-reported by Google (2025a, 2025b). Open-source security-tuned LLMs include the Cisco Foundation-Sec-8B family (Kassianik et al. (2025); Yang et al. (2026)) and the community-driven

WhiteRabbitNeo-V2-70B (WhiteRabbitNeo (2024)). RoBERTa-base CVE→CWE is from Mosievskiy (2026); their reported 75.6% strict accuracy is on the CTI-Bench RCM benchmark, but the exact split (CTI-RCM-2024 only, or both CTI-RCM-2024 and CTI-RCM-2021 combined) is not separately reported and we did not re-run their model on our split. "Ours" is the locked recipe evaluated in this work on the CTI-RCM-2024 split. The dashed red line marks the published GPT-4 baseline; the locked recipe exceeds it by 13.1 percentage points and is statistically tied with Google's best closed security-tuned models.

The locked pipeline exceeds the published GPT-4 baseline by 13.1 percentage points and the fine-tuned RoBERTa baseline by 9.5 percentage points, while running fully locally on a single consumer GPU and using only public data sources. The framing for an external reader is important: this is "local 7B open-source model plus retrieval over public NVD and MITRE data" versus "proprietary frontier model with no retrieval." Security teams routinely have access to NVD, so this comparison reflects a realistic deployment scenario, but it is not a claim that the 7B model is intrinsically stronger than GPT-4.

Table 1. CTI-Bench RCM leaderboard (full numeric comparison)

Model	Params	Type	cti-rcm	Source
Sec-Gemini v1 (Google)*	—	closed	~86%	Google (2025a)
Ours (locked recipe)	**7B**	**open**	**85.1%**	**this work**
SecLM (Google)*	—	closed	~85%	Google (2025b)
RoBERTa-base CVE→CWE	125M	open	75.6%	Mosievskiy (2026)
Foundation-Sec-8B-R (Cisco)	8B	open	75.3%	Yang et al. (2026)
GPT-4	~1.7T	closed	72.0%	Alam et al. (2024)
Foundation-Sec-8B (Cisco)	8B	open	72.0%	Kassianik et al. (2025)
WhiteRabbitNeo-V2-70B	70B	open	71.1%	Kassianik et al. (2025)
Foundation-Sec-8B-I (Cisco)	8B	open	70.4%	Yang et al. (2026)
Llama-Primus-Base (Trend Micro)	8B	open	67.8%	Yu et al. (2025)
GPT-3.5	~175B	closed	67.2%	Alam et al. (2024)
Gemini 1.5	—	closed	66.6%	Alam et al. (2024)
LLaMA3-70B	70B	open	65.9%	Alam et al. (2024)
LLaMA3-8B	8B	open	44.7%	Alam et al. (2024)

* Sec-Gemini v1 and SecLM scores self-reported by Google in technical blog posts; not independently evaluated on a public split. All non-self-reported numbers above were aggregated by Mosievskiy (2026) on the CTI-Bench RCM benchmark. Our 85.1% was measured on the 1,000-query CTI-RCM-2024 split using the locked recipe described in this report.

The leaderboard puts the locked recipe at the top of the open-source category and statistically tied with Google's best closed security-tuned models (Sec-Gemini v1 at ~86%, SecLM at ~85%), substantially ahead of every other open-source baseline including the fine-tuned RoBERTa CVE→CWE classifier and the Cisco Foundation-Sec-8B family. The result demonstrates that a carefully engineered retrieval pipeline around a small open-source LLM can match models trained specifically on cybersecurity corpora without itself requiring domain-specific pretraining or fine-tuning.

Table 2. Ablation of retrieval-pipeline stages (1,000-query full eval)

Configuration	Total	NVD-mapped (903)	NVD-unmapped (97)
Hybrid retrieval + bridge only (baseline)	843/1000 (84.3%)	782/903 (86.6%)	61/97 (62.9%)
+ Official CWE phrase selector	843/1000 (84.3%)	775/903 (85.8%)	68/97 (70.1%)
+ Cross-encoder reranking	845/1000 (84.5%)	775/903 (85.8%)	70/97 (72.2%)
+ HyDE expansion with cross-encoder filter	849/1000 (84.9%)	779/903 (86.3%)	70/97 (72.2%)
+ Tightened k-NN confidence (θ = 0.90, margin = 1.50)	**851/1000 (85.1%)**	**779/903 (86.3%)**	**72/97 (74.2%)**

The ablation isolates each stage's contribution at full-benchmark scale. The phrase selector lifts unmapped accuracy by 7 cases (62.9 → 70.1%) without changing the mapped-subset score meaningfully, because mapped CVEs already win via the authoritative NVD bridge before phrase-selector logic engages. Cross-encoder reranking adds 2 more unmapped cases at the cost of zero mapped cases, consistent with its role in resolving sibling/parent CWE confusion in the description-only path. HyDE plus the cross-encoder filter recovers 4 mapped cases and holds unmapped flat at this configuration. Finally, tightening the k-NN confidence threshold and margin recovers 2 more unmapped cases by preventing wrong-but-confident k-NN votes from stripping the correct CWE from context. The cumulative lift is 8 cases on the full benchmark (843 → 851) and 11 cases on the unmapped subset (61 → 72).

Table 3. Final locked-recipe breakdown

Subset	Cases	Accuracy
All queries	851 / 1000	85.1%
NVD-mapped subset	779 / 903	86.3%
NVD-unmapped subset	72 / 97	74.2%

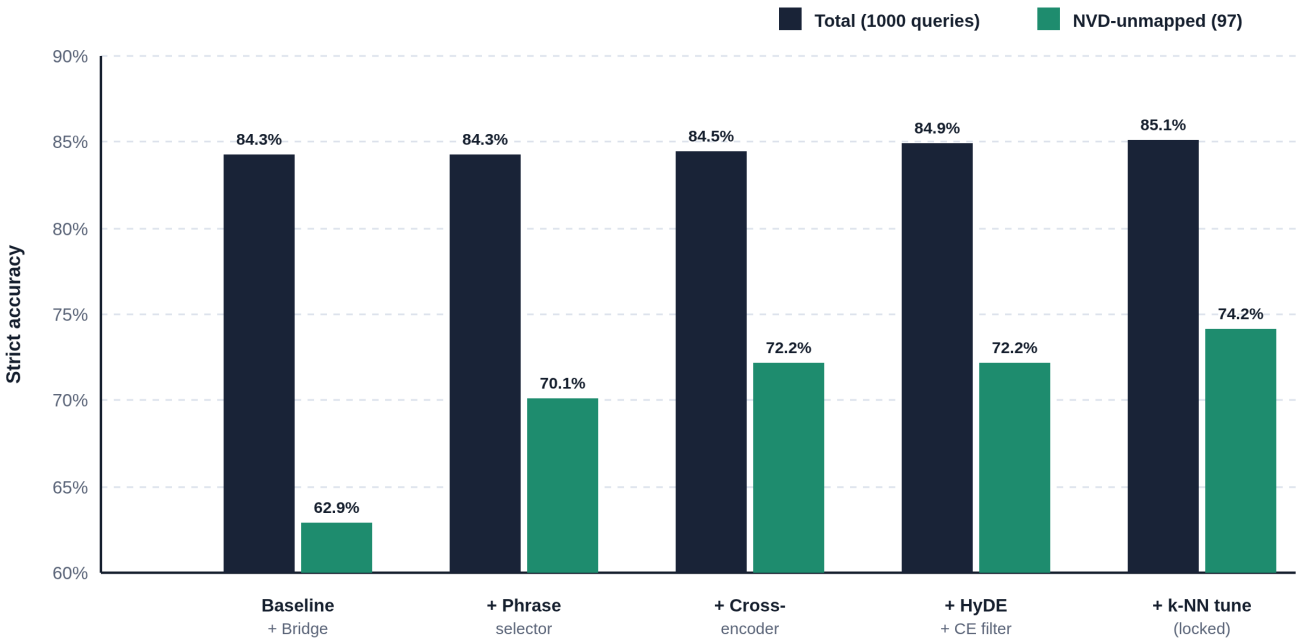
The split between mapped and unmapped subsets is informative for understanding what the pipeline is doing. The mapped subset (903 queries) is solved primarily by the bridge-injection path: the embedding-based retrieval finds the matching CVE in the knowledge base, and the NVD-assigned CWE is injected into the LLM context. The unmapped subset (97 queries) is solved by the description-based reasoning path, where retrieval surfaces CWE candidates and the LLM selects from them. The unmapped subset is the more difficult and is where the multi-stage reranking matters most; it improved from 62.9% baseline to 74.2% final, while the mapped subset moved only slightly because it was already near the retrieval-bound ceiling.

Failure-mode analysis

A diagnostic pass over the 25 unmapped failures captured at the cross-encoder-only configuration showed two roughly equal failure modes. Approximately 40% of failures are language-model errors: the correct CWE was present in the final context, but the model selected a sibling or near-miss CWE (for example, CWE-77 versus CWE-78 in command injection, or CWE-119 versus CWE-190 in memory-buffer cases). Approximately 60% are retrieval misses: the correct CWE was not present in the final context, often for abstract weakness categories (CWE-755 Improper Handling of Exceptional Conditions, CWE-668 Exposure of Resource to Wrong Sphere) whose definitions share little surface vocabulary with concrete CVE prose. This split informs future-work directions: the language-model-error fraction can in principle be addressed by retrieval-augmented fine-tuning of the language model, and the retrieval-miss fraction may benefit from domain-adapted embeddings or domain-balanced corpus augmentation.

The figure below visualizes the per-stage contribution across the five cumulative configurations.

CTI-RCM Accuracy by Pipeline Stage



Cumulative ablation. Each stage builds on the previous. 5 conditions evaluated on the full 1000-query CTI-Bench RCM split.

CTI-RCM strict accuracy by pipeline stage. Dark bars: total accuracy on the full 1000-query benchmark; green bars: accuracy on the 97-CVE NVD-unmapped subset. The mapped subset (903 CVEs) stays near-ceiling at 86.3% across all configurations, so all the headline gain on the unmapped subset translates to roughly +0.2-0.6 points on the total score per stage.

Project Timeline

Phase	Tasks	Deliverables	Estimated Time
1. Problem Definition & Literature Review	Identify CTI-RCM as primary benchmark; survey RAG, hybrid retrieval, cross-encoder rerank, HyDE, RAFT, CTI-Bench, ThreatZoom; formulate research question	Problem statement, related work section	Week 1–2
2. System Architecture & Knowledge Base	Design hybrid-retrieval pipeline; build CWE, CAPEC, ATT&CK, CVE chunk builders; extract cross-source graph edges	Block diagram, chunked KB (~336k chunks)	Week 3
3. Hybrid Retrieval Implementation	Implement BM25 + BGE-small dense retrieval, hybrid scoring, top-K filter; vectorize for throughput	`better_rag.py` core retriever	Week 4
4. Graph Expansion & Bridge Injection	Wire ATT&CK→CAPEC, CAPEC→CWE, CVE→NVD `cwe_ids` bridges; add bridge-strip rule for authoritative mappings	Working mapped-CVE path	Week 5
5. Unmapped-CVE Fallback	Implement weighted k-NN CWE voting over 150k mapped CVEs; calibrate threshold and margin	Working unmapped-CVE path	Week 6
6. Multi-Stage Reranking	Implement official CWE phrase selector from MITRE XML; integrate cross-encoder rerank (`bge-reranker-`	Multi-stage reranker	Week 7–8

Phase	Tasks	Deliverables	Estimated Time
	`base`); add HyDE with CE filter		
7. CTI-RCM Eval Harness	Build `eval_rcm.py` per CTI-Bench protocol; support concurrent workers; add `--debug-failures` mode for diagnostic capture	Eval harness, ablation runner	Week 9
8. Ablations & Hyperparameter Tuning	Run paired ablations for each stage at full 1000-query scale; sweep k-NN threshold/margin deterministically	Ablation tables (Table 2), locked recipe	Week 10
9. Failure-Mode Diagnostic Pass	Classify residual failures into retrieval misses vs. language-model errors; identify future-work directions	Failure-mode breakdown, future-work plan	Week 11
10. Writing, Formatting & Final Report	Write methodology, system design, implementation, results; produce tables and figure; assemble references	Complete final report	Week 12–13

References

- Aghaei, E., Niu, X., Shadid, W., & Al-Shaer, E. (2020). ThreatZoom: CVE2CWE using hierarchical neural network. *SecureComm 2020*.
- Aghaei, E. (2025). SecureBERT 2.0: Advanced language model for cybersecurity intelligence. *arXiv preprint*.
- Alam, M. T., Bhusal, D., Park, Y., & Rastogi, N. (2024). CTIBench: A benchmark for evaluating LLMs in cyber threat intelligence. *NeurIPS 2024*.
- Bayer, M., Kuehn, P., Shanehsaz, R., & Reuter, C. (2024). CySecBERT: A domain-adapted language model for the cybersecurity domain. *ACM Transactions on Privacy and Security*.

- Chen, S., Niu, S., & McAuley, J. (2023). DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. *RAID 2023*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient finetuning of quantized LLMs. *NeurIPS 2023*.
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT 2019*.
- Ethayarajh, K., Xu, W., Muennighoff, N., Jurafsky, D., & Kiela, D. (2024). KTO: Model alignment as prospect theoretic optimization. *arXiv preprint*.
- Gao, L., Ma, X., Lin, J., & Callan, J. (2022). Precise zero-shot dense retrieval without relevance labels (HyDE). *arXiv preprint*.
- Google. (2025a). Google announces Sec-Gemini v1, a new experimental cybersecurity model. *Google Online Security Blog* (April 4, 2025).
- Google. (2025b). Fueling AI innovation in SecOps products: The SecLM platform and Sec-Gemini research pipeline. *Google Cloud Community Blog*.
- Hanif, H., & Maffeis, S. (2022). VulBERTa: Simplified source code pre-training for vulnerability detection. *IJCNN 2022*.
- Hong, J., Lee, N., & Thorne, J. (2024). ORPO: Monolithic preference optimization without reference model. *EMNLP 2024*.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. *ICLR 2022*.
- Kalajdzievski, D. (2023). A rank stabilization scaling factor for fine-tuning with LoRA. *arXiv preprint*.
- Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W. (2020). Dense passage retrieval for open-domain question answering. *EMNLP 2020*.
- Kassianik, P., et al. (2025). Llama-3.1-FoundationAI-SecurityLLM-Base-8B technical report. *arXiv preprint* (Cisco Foundation AI).
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., & Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention (vLLM). *SOSP 2023*.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *NeurIPS 2020*.
- Lin, J., Nogueira, R., & Yates, A. (2021). Pretrained transformers for text ranking: BERT and beyond. *Synthesis Lectures on Human Language Technologies*.
- Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C. F., Cheng, K.-T., & Chen, M.-H. (2024). DoRA: Weight-decomposed low-rank adaptation. *ICML 2024*.
- MITRE Corporation. (2024a). Common Weakness Enumeration (CWE) version 4.x.
- MITRE Corporation. (2024b). Common Attack Pattern Enumeration and Classification (CAPEC).

- Moreira, G., Osmulski, R., Xu, M., Ak, R., Schifferer, B., & Oldridge, E. (2024). NV-Retriever: Improving text embedding models with effective hard-negative mining. *arXiv preprint*.
- National Institute of Standards and Technology. (2024). National Vulnerability Database.
- Nogueira, R., & Cho, K. (2019). Passage re-ranking with BERT. *arXiv preprint*.
- Qwen Team. (2024). Qwen2.5 technical report. *arXiv preprint*.
- Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). Direct preference optimization: Your language model is secretly a reward model. *NeurIPS 2023*.
- Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence embeddings using siamese BERT-networks. *EMNLP 2019*.
- Robertson, S., & Zaragoza, H. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 333–389.
- Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint*.
- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E., Schärli, N., & Zhou, D. (2023). Large language models can be easily distracted by irrelevant context. *ICML 2023*.
- Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). MITRE ATT&CK: Design and philosophy. *MITRE Technical Report*.
- Touvron, H., Lavril, T., Izacard, G., et al. (2023). Llama: Open and efficient foundation language models. *arXiv preprint*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *NeurIPS 2017*.
- WhiteRabbitNeo. (2024). WhiteRabbitNeo-V2 cybersecurity LLM (Llama-3.1 8B / 70B). *Hugging Face model card*.
- Xiao, S., Liu, Z., Zhang, P., & Muennighoff, N. (2024). C-Pack: Packaged resources to advance general Chinese embedding (BGE). *SIGIR 2024*.
- Xiong, L., Xiong, C., Li, Y., Tang, K.-F., Liu, J., Bennett, P. N., Ahmed, J., & Overwijk, A. (2021). Approximate nearest neighbor negative contrastive learning for dense text retrieval (ANCE). *ICLR 2021*.
- Yang, A., et al. (2026). Llama-3.1-FoundationAI-SecurityLLM-Reasoning-8B technical report. *arXiv preprint* (Cisco Foundation AI).
- Yu, Y.-C., et al. (2025). Primus: A pioneering collection of open-source datasets for cybersecurity LLM training. *arXiv preprint* (Trend Micro).
- Zhang, T., Patil, S. G., Jain, N., Shen, S., Zaharia, M., Stoica, I., & Gonzalez, J. E. (2024). RAFT: Adapting language model to domain specific RAG. *arXiv preprint*.
- Zhao, W., Zheng, Y., Wang, S., Lakshmanan, L. V. S., Ren, X., & Chen, J. (2024). Dense X retrieval: What retrieval granularity should we use? *EMNLP 2024*.
- Mosievskiy, N. (2026). Fine-tuning RoBERTa for CVE-to-CWE classification: A 125M parameter model competitive with LLMs. *arXiv preprint*.

- Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR 2024 Oral*.
- Izacard, G., & Grave, E. (2021). Leveraging passage retrieval with generative models for open domain question answering. *EACL 2021*.
- Khattab, O., & Zaharia, M. (2020). ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. *SIGIR 2020*.
- Brown, T. B., Mann, B., Ryder, N., et al. (2020). Language models are few-shot learners. *NeurIPS 2020*.
- Touvron, H., Martin, L., Stone, K., et al. (2023). Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint*.
- OpenAI. (2023). GPT-4 technical report. *arXiv preprint*.
- Yao, S., Yu, D., Zhao, J., Shafran, I., Griffiths, T. L., Cao, Y., & Narasimhan, K. (2023). Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS 2023*.